

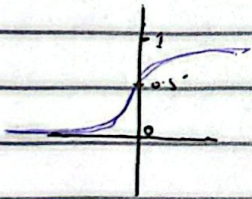
⇒ Activation Functions:

⇒ (1) Sigmoid Function:

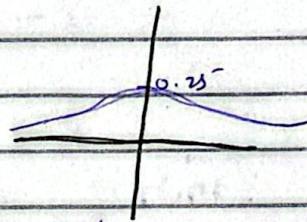
$$\star \sigma(z) = \frac{1}{1 + e^{-z}}$$

★ It ranges b/w: 0-1

★ Derivative of sigmoid function ranges b/w: 0-0.25



Sigmoid



Derivative of Sigmoid

Disadvantages:

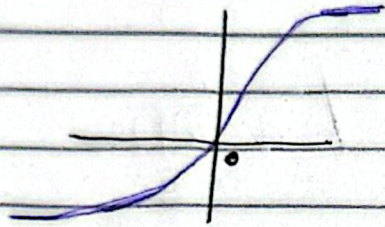
★ Using Sigmoid in Deep neural Network causes vanishing gradient problem.

★ Function output is not zero center (zero centered Function, updates weights efficiently).

Advantages:

- Suitable for Binary classification
- clear prediction i.e. very close 1 or 0-

Zero-Centered:



- This is a zero centered function
- 0-centered means The mean is zero.
- Efficient weight updation will happen.

⇒ When we apply Standard-Scaler it makes the value zero centered i.e. mean is zero & std is 1

$$x \longrightarrow x$$

↳ (2) Tanh Function:

$$\star \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

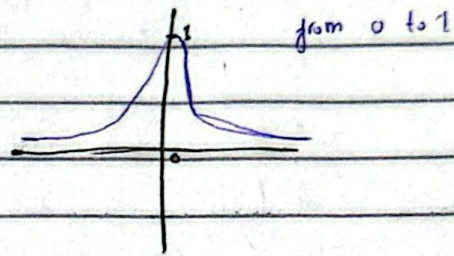
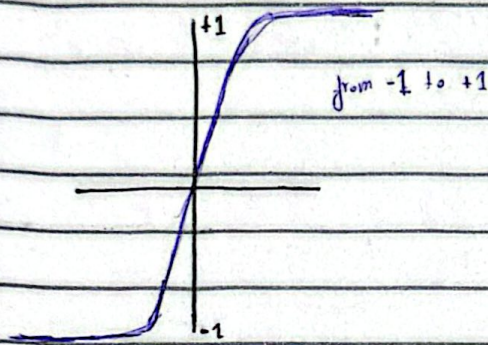
or

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\therefore z = \sum_{i=1}^n w_i^T x_i + b$$

* Tanh ranges b/w : $-1 - +1$

* Derivative ranges b/w : $0 - 1$



Tanh

Derivative of Tanh.

* Tanh is zero centric.

* For medium size neural network we won't have vanishing gradient problem.

*** But in Deep neural Network we may face some vanishing gradient problem as derivative ranges b/w 0-1.

From smaller $\leftarrow 0.7 \times 0.01 \times 0.6 \times 0.2 \times 0.025 \times 0.5$
value $0 \text{ --- } 0 \text{ --- } 0 \text{ --- } 0 \text{ --- } 0$

↳ (3): ReLU Activation

Function:

$$* \text{Relu} = \max(0, x)$$

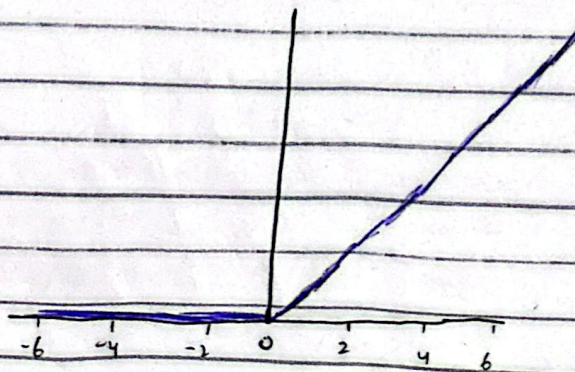
→ it is max of $(0, x)$

→ if the " x " is positive number
then it will come as " x "

→ if The " x " is any negative number
then it will become "zero"

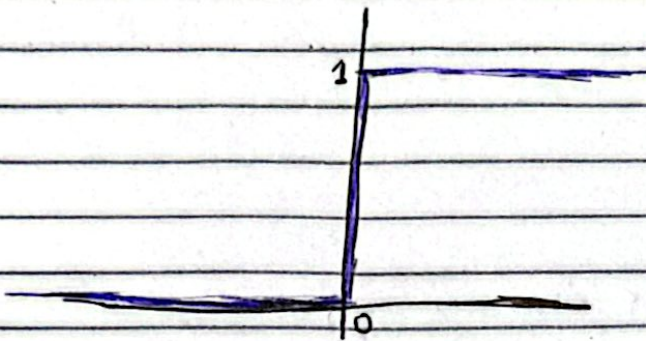
e.g if $x = 2$ Then it will come as 2

if $x = -6$ Then it will become 0



* Derivative of Relu : The answer is either zero or one.

→ if positive number == 1
→ if negative number == 0



⇒ if Relu output is 1 ⇒ weight updation will happen.

(it solves The negligible change problem that causes vanishing gradient, that was in sigmoid & Tanh).

⇒ if Relu output is 0 ⇒ this creates dead neuron.

$$w_{\text{new}} = w_{\text{old}} - \eta \left[\frac{\partial \text{Loss}}{\partial w_{\text{old}}} \right]$$

zero b/c relu output is zero

$$w_{\text{new}} = w_{\text{old}} - \eta(0) \Rightarrow w_{\text{new}} = w_{\text{old}}$$

*** if z is +ve $\Rightarrow \frac{\partial (\text{Relu}(z))}{\partial (z)} = 1$

if z is -ve $\Rightarrow \frac{\partial (\text{Relu}(z))}{\partial (z)} = 0$

→ if $x = -5$ then $\text{Relu}(-5) = 0$ & $\frac{\partial(\text{Relu}(x))}{\partial(x)} = \frac{\partial(0)}{\partial(-5)} = 0$

Advantages

- Solve gradient vanishing problem
- $\max(0, x) \rightarrow$ computation is fast

Disadvantages

- Makes dead neuron when Relu output is zero.
- Not zero centered (as you will get either zero or +ve).

→ (4): Leaky Relu & Parametric Relu:

- * The dead neuron problem is solved using Leaky Relu & parametric Relu

⇒ In Leaky Relu we add a small constant in the function.

Normal Relu = $\max(0, x)$

** Leaky Relu = $\max(\alpha x, x)$

$\therefore \alpha$ is the const that we add.

Suppose $\alpha = 0.01$

$\Rightarrow$ at $x = -5^-$, so:

$$\text{Leaky ReLU} = \min(0.01 \times (-5^-))$$

$$\text{Leaky ReLU} = -0.05^-$$

* See The -ve value didn't become "zero" like Normal ReLU where -ve becomes "zero".

So $f(z) = (\alpha \times x, x)$

$$\begin{aligned} f(-5^-) &= 0.01 \times (-5^-) \\ \Rightarrow f(-5^-) &= -0.05^- \end{aligned}$$

*** Don't make ~~see~~ Negative Values Completely zero, keep tiny fraction of it.

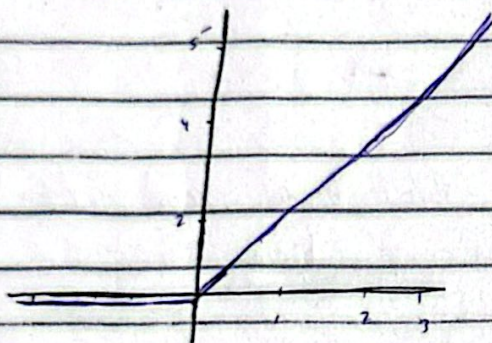
$\Rightarrow$ As Negative Values don't become zero, so it will have some slope (Non-zero).

$$\Rightarrow \text{if } z = 5^-, \text{ReLU}(5^-) = 5^- \Rightarrow \frac{\partial(\text{Leaky ReLU}(5^-))}{\partial(5^-)} = 1$$

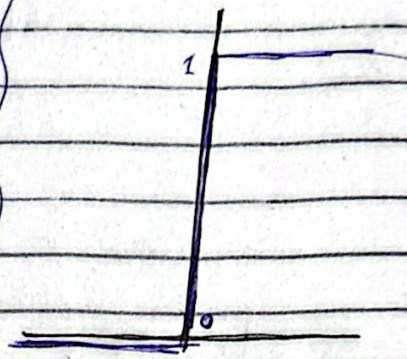
$$\Rightarrow \text{if } z = -5^-, \text{Leaky ReLU}(-5^-) = -0.05^- \Rightarrow \frac{\partial(\text{Leaky ReLU}(-0.05^-))}{\partial(-0.05^-)} = \text{Some value}$$

*** As it will have some value & Not entirely zero, so it solves the Dead Neuron problem in backward propagation.

⇒ In Leaky Relu The constant (α) is fixed (constant) & do not change



Leaky Relu



→ Derivative of Leaky Relu

→ observe its Not zero & little in the -ve side

→ slope is not zero rather would have some -ve value for -ve z .

→ Parametric Relu:

$$f(z) = \max(\alpha z, z)$$

or

$$f(x) = \max(\alpha x, x)$$

* In parametric Relu " α " is not constant & can be changed.

⇒ here " α " is a hyper parameter & we can change its value.

⇒ This is the only difference b/w leaky & parametric ReLU everything else is same.

Advantages:

* Leaky ReLU has all the advantages of ReLU

* Solve The dead neuron problem.

Disadvantage: Not zero centric $\Rightarrow$ Weight updation is not efficient.

↳ 5): ELU (Exponential
Linear Unit):

* This Activation function is also Designed to Solve The dead neuron problem of ReLU.

$$\Rightarrow \text{ELU}(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha(e^z - 1) & \text{otherwise.} \end{cases}$$

⇒ if $x > 0$:
at $x > 0$ ELU behaves exactly like ReLU

$$\rightarrow ELU(x) = x$$

if $x = 5$
 $ELU(5) = 5$

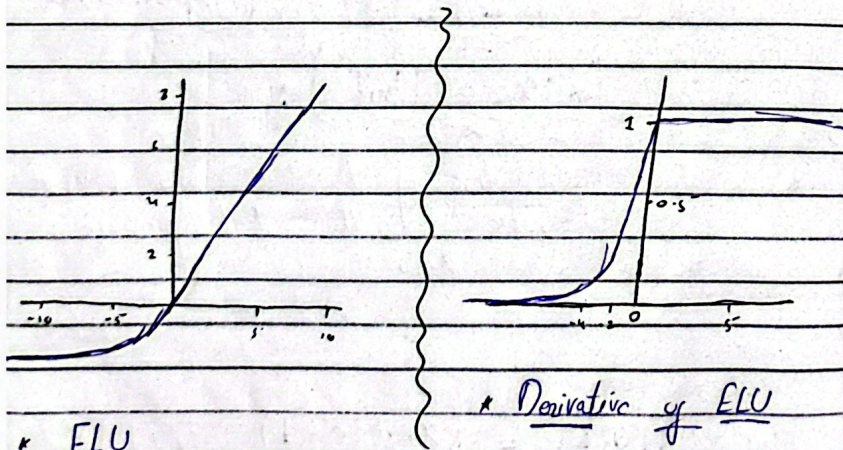
⇒ if $x < 0$:

suppose " α " = 1 & $x = -1$

$$ELU(-1) = e^{-1} - 1$$

$$ELU(-1) \approx -0.6321$$

↳ It will have slope



* ELU

* Derivative of ELU

• for -ve value the derivative is +ve

* For negative inputs it uses exponential curve, whose derivative becomes positive.

Advantages:

Disadvantages

* No Dead ReLU Neuron

* Slightly more

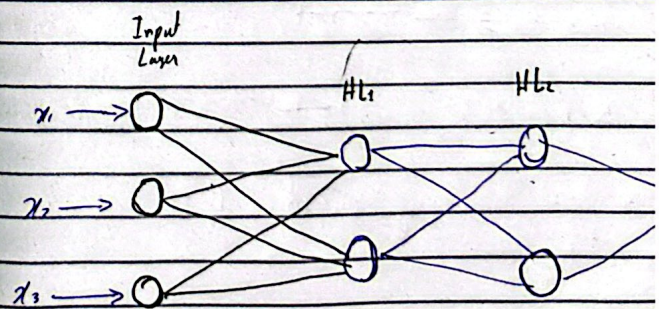
* Zero Centered

computation.

↳ 6) SoftMax Act

Function:

* Used in The Output Layer Network for Multi-class classification problem.



output the Activation function values from

* For negative inputs it uses exponential curve, whose derivative becomes positive.

Advantages:

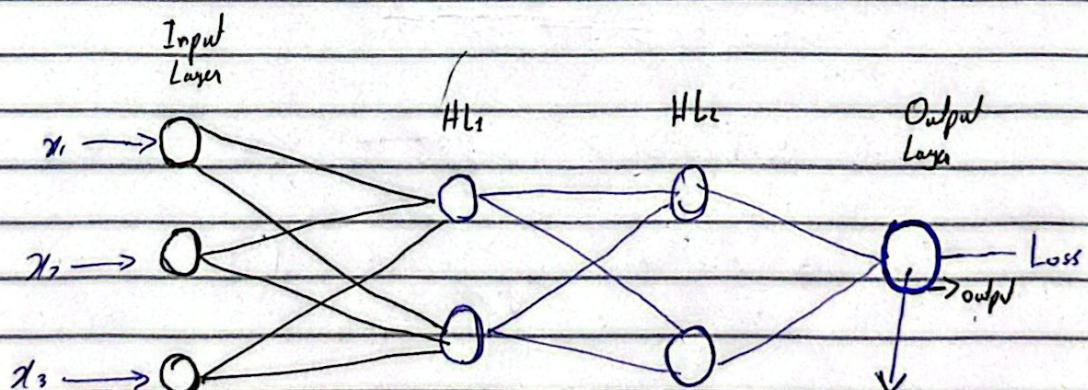
- * No Dead ReLU Neuron
- * Zero Centric.

Disadvantages

- * slightly more computational.

↳ 6): SoftMax Activation Function:

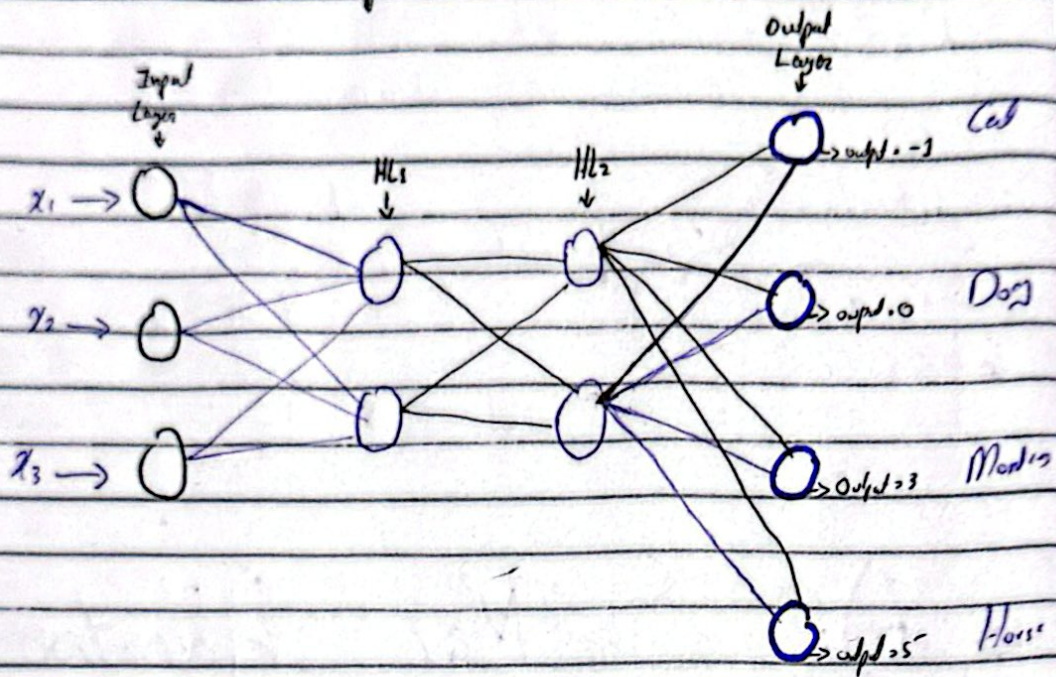
* Used in the Output Layer of Neural Network for Multi-class classification problem.



** In binary classification, output layer uses sigmoid Activation function as it transforms values between 0 & 1

* In output layer we use 4 different outputs

⇒ Multi-Class Classification: 4-categories [cat, dog, monkey, horse]



* So Here we can't use sigmoid, so we use Softmax, activation function

$$\Rightarrow \boxed{\text{Softmax} = \frac{e^{y_i}}{\sum_{k=0} e^{y_k}}}$$

∴ $i = \text{Node}$ { 1st Node $\Rightarrow e^{y_1}$, 2nd Node $\Rightarrow e^{y_2}$

∴ $k = \text{Nmbre of output Nodes}$

∴ $y^1 = \text{output of 1st Node}$

∴ $y^2 = \text{output of 2nd Node}$
or

for Cat = for cat $\Rightarrow y^1 = y^2 = \text{output} = -1$

$$= \frac{e^{-1}}{e^{-1+0+3+5}} \Rightarrow 0.00033$$

for Dog: $\frac{e^0}{e^{-1+0+3+5}} \Rightarrow 0.0024$

for Monkey: $\frac{e^3}{e^{-1+0+3+5}} \Rightarrow 0.0183$

for Horse: $\frac{e^5}{e^{-1+0+3+5}} \Rightarrow 0.1353$

$\Rightarrow$ Now find probability of output.

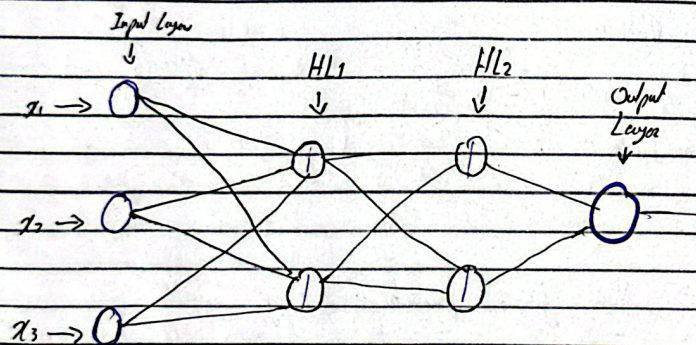
$$P(\text{horse}) = \frac{0.1353}{0.00033 + 0.0024 + 0.0183 + 0.1353} \approx 86\%$$

$$P(\text{Dog}) = \frac{0.0024}{0.00033 + 0.0024 + 0.0183 + 0.1353} \approx 4\%$$

$\Rightarrow$ find for other too

* Horse is highest probability, so output is Horse.

⇒ Which Activation Function to Use When?



Hidden Layer:

* Never use "Sigmoid" or "Tanh" in Hidden Layers → They Lead to Vanishing gradient problem.

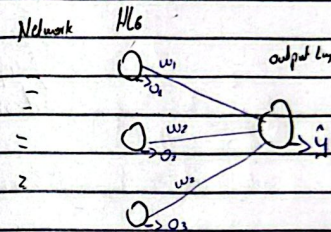
* In Hidden Layers use ReLU or Variants of ReLU.

Output Layer:

* If Binary classification use "Sigmoid"
* If Multiclass classification "Softmax"

* In Regression we don't use Activation Function in the output layer

* The output Neuron Takes all the outputs given by previous hidden layers multiplies each by its own weight & add them.

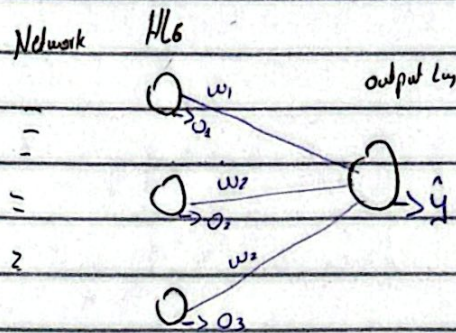


Suppose

$$\hat{y} = w_1 O_1 + w_2 O_2 + w_3 O_3$$

* In Regression We don't use Activation Function in the output Layer.

* The output Neuron Takes, all the outputs given by previous hidden Layer multiplies each by its own weight & add them.



Suppose

So

$$\hat{y} = w_1 o_1 + w_2 o_2 + w_3 o_3$$

⇒ Cost Function vs Loss Function

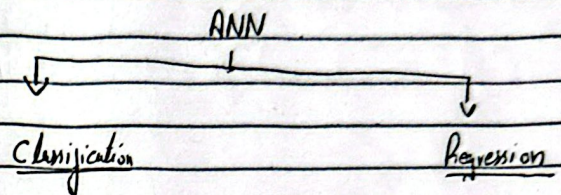
Loss Function: For every data point/input we do forward propagation find error $(y - \hat{y})$ & then to minimize it backward propagation

$$MSE = (y - \hat{y})^2$$

Cost Function: We send all the inputs as once find error for each & add the errors of all the points $(\sum_{i=1}^n (y - \hat{y})^2)$ & then to minimize it, do backward propagation once.

$$MSE = \sum_{i=1}^n (y - \hat{y})^2$$

Note: we have studied these function in Linear Regression.



- | | |
|--|--|
| (1) Cross Entropy | (1) Mean Squared error |
| ↓
• binary cross entropy
• categorical cross entropy
• sparse categorical cross entropy | (2) Mean Absolute error
(3) Root Mean Squared error
(4) Huber Loss |

For Regression

⇒ Mean Squared Error (MSE):

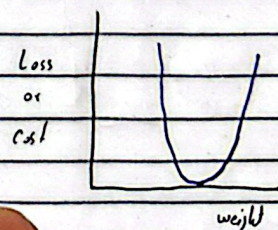
$$* \text{Loss}_{ji} = (y - \hat{y})^2, \quad \text{cost}_{ji} = \sum_{i=1}^n (y - \hat{y})^2$$

Advantages

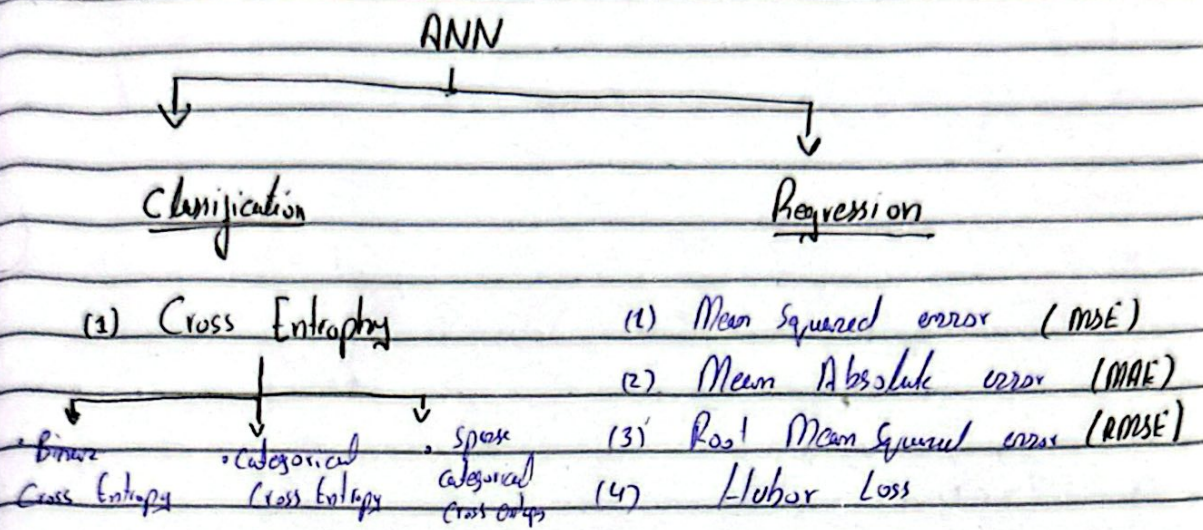
- MSE is differentiable
- It has one local or global minima
- It converges faster

Disadvantages

- Not Robust
- sensitive



Note: We have studied these function in Linear Regression.



For Regression

↳ Mean Squared Error (MSE):

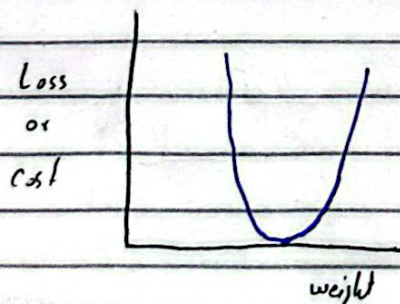
$$\star \text{ Loss}_{jn} = (y - \hat{y})^2 \quad , \quad \text{cost}_{jn} = \sum_{i=1}^n (y - \hat{y})^2$$

Advantages

- MSE is differentiable
- It has one local or global minima
- It converges faster

Disadvantages

- Not Robust to outliers



↳ Mean Absolute Error (MAE) :

$$x \text{ Loss } f_n = |y - \hat{y}| \quad , \quad \text{cost } f_n = \frac{1}{n} \sum_{i=1}^n |y - \hat{y}|$$

→ Robust to outliers

→ Convergence takes time in MAF

↳ Flavor Loss:

Combination of "MSF" & "MAE"

$$\text{Cost } J_n = \begin{cases} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) & ; \text{ if } |y_i - \hat{y}_i| \leq \delta \\ \frac{|y_i - \hat{y}_i| - \delta}{2} & ; \text{ otherwise.} \end{cases}$$

$\therefore \delta = \text{hyper parameter (Threshold)}$

- So if $|y - \hat{y}| \leq S(\text{Threshold}) \Rightarrow$ Means True

is no outliers, so we use MSE, if greater means outliers then we use MAE.

↳ RMSE:

Root	MSE
------	-----

$$\cos \theta_n = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

For Classification

Cross Entropy:

- Binary Cross Entropy { Binary Classification }
- Categorical Cross Entropy { Multi-class classification }
- Sparse Categorical Cross Entropy { Multi-class classification }

↳ a) Binary Cross Entropy:

$$* \text{ Loss} = -y * \log(\hat{y}) - (1-y) * \log(1-\hat{y})$$

$$\text{Loss} = (-\log(1-y)) \quad \text{if } y = 0$$

Note: Same Loss we learn in Logistic Pa

↳ RMSE :

Root MSE

$$\text{Cost fn} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2}$$

x → x

For Classification

Cross Entropy :

- Binary Cross Entropy { Binary Classification }
- Categorical Cross Entropy { Multi-class class }
- Sparse Categorical Cross Entropy { Multi-class classification }

↳ a) Binary Cross Entropy :

$$* \text{ Loss} = -y * \log(\hat{y}) - (1-y) * \log(1-\hat{y})$$

$$\text{Loss} = \begin{cases} -\log(1-\hat{y}) & \text{if } y=0 \\ -\log(\hat{y}) & \text{if } y=1 \end{cases}$$

Note: Same Log loss we learned in Logistic Regression

→ b) Categorical Cross Entropy:

* Used for Multi-class classification

Dataset:

x_i	y_i^1	y_i^2	y_i^3	O/P
$x_1 = 2$	3	4		Good
$x_2 = 5$	6	7		Bad
$x_3 = 8$	9	10		Neutral
				↓ OHE

* It will convert the output/all the categories in OHE.

	Good ($j=1$)	Bad ($j=2$)	Neutral ($j=3$)
x_1 :	1	0	0
x_2 :	0	1	0
x_3 :	0	0	1

$$* \text{Loss}(x_i, y_i) = - \sum_{j=1}^C y_{ij} * \ln(\hat{y}_{ij})$$

- ∴ C = number of categories
- ∴ i = data points ($i = 1, n$), i = row
- ∴ j = Category
- ∴ $\hat{y}_{ij}$ = Prediction

$$* \text{Row } i = y_i = [y_{i1}, y_{i2}, y_{i3}]$$

$$\text{Row } j = [y_{j1}, y_{j2}, y_{j3}]$$

$$\vdots$$

$$n$$

$$y_{ij} = \begin{cases} 1 & ; \text{ if the element is in the class} \\ 0 & ; \text{ if element not in the class} \end{cases}$$

$$*** \text{ Prediction} = \hat{y}_{ij} \Rightarrow \text{Softmax Act fn} \Rightarrow \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

⇒ softmax as its Multi-class comparison.

⇒ Softmax give us probabilities.

So:

$$\hat{y}_{ij} = \text{Probabilities}$$

$$\Rightarrow [0.2, 0.3, 0.5]$$

* Also give probabilities of other categories

$$\text{for } \begin{matrix} \text{row 1} \\ \text{row 2} \\ \vdots \\ \text{row } n \end{matrix} = y_{ij} = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ \vdots & \vdots & \vdots \\ y_{n1} & y_{n2} & y_{n3} \end{bmatrix}$$

$$y_{ij} = \begin{cases} 1 & ; \text{ if the element is in the class} \\ 0 & ; \text{ if element not in the class} \end{cases}$$

$$*** \text{ Prediction} = \hat{y}_{ij} \Rightarrow \text{Soft max Act fn} \Rightarrow \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$\Rightarrow$ soft max as its Multi-class classification.

$\Rightarrow$ Soft max give us probabilities.

So:

$$\hat{y}_{ij} = \text{Probabilities}$$

$$\Rightarrow [0.2, 0.3, 0.5]$$

* Also give probabilities of other categories.

↳ 3): Sparse Categorical Cross Entropy.

⇒ here everything is same but in output we get the index of highest probable category.

$$\hat{y}_{ij} = \begin{matrix} \text{index: } 0 & 1 & 2 \\ \begin{bmatrix} 0.2 & 0.3 & 0.5 \end{bmatrix} \end{matrix}$$

⇓

2nd index ⇒ output.

Disadvantage:

* Losing info about other category ~~entropies~~ probabilities.

⇒ Which Loss Function to Use

Best Combinations:

<u>Hidden Layer</u>	<u>Output Layer</u>	<u>Problem Statement</u>
1) Relu	Sigmoid	Binary Classification
2) Relu or Variants	Soft-max	Multi-class Classification
3) Relu or Variants	Linear	Regression

⇒ Which Loss Function to Use When?

Best Combinations:

	<u>Hidden Layers</u>	<u>Output Layer</u>	<u>Problem Statement</u>	<u>Loss function</u>
1)	Relu	Sigmoid	Binary classif.	Binary Cross Entropy.
2)	Relu or Variants	Soft-max	Multi-class classification	Categorical or Sparse CE
3)	Relu or Variants	Linear	Regression	MSE, MAE, RMSE Huber loss.